
Subject: FW: Medoc - - - Discussion of External Control options
Date: Monday, April 27, 2026 at 5:21:08 PM Eastern Daylight Time
From: Ryan Bell, Ph.D.
To: Eric Wang
Attachments: image001.png, image002.jpg, Matlab_External_Control_Code_Example.zip

From: medoc@mindspring.com <medoc@mindspring.com>
Date: Monday, April 27, 2026 at 5:07 PM
To: Ryan Bell, Ph.D. <ryan.bell@duke.edu>
Subject: FW: Medoc - - - Discussion of External Control options

From: medoc@mindspring.com <medoc@mindspring.com>
Sent: Tuesday, April 14, 2026 9:56 AM
To: 'Ryan Bell, Ph.D.' <ryan.bell@duke.edu>
Subject: Medoc - - - Discussion of External Control options

Hi Ryan,

Sorry for the delay in responding to your email of last week about our External Control for TSA2 (and TSA2 AIR). To address your question....below is an email constructed by our CTO. Gilad does a better job of describing the options than can I. I would also add that Gilad mentioned he will be happy to set a zoom at some point to assist/review options/address questions, etc. Let me know if any questions, as well as if you decide the above mentioned zoom with Gilad would be of interest.

Best regards,

Phil



We have two methods to control the device using customized code

1. The "old" method, where the Matlab/Python/E-Prime code opens a socket over TCP/IP to the network interface card of the PC running our MMS software. There is an IP address assigned which you would see in the External Control menu, and a dedicated port number. This method suffers from problems related to TCP/IP, where idle times may lead to timeouts and disconnects, and missed triggers. The problem may be resolved by periodically sending a packet to retrieve a response from the device/MMS, however the functionality of this method is limited, you need to predefine programs and simply activate them with Stop/Start commands. There is a way to dynamically change a destination temperature in specific program types (VAS Search, Search), but depending on the complexity of your experiment setup, it might be easier to use method 2 below. In any case I've attached the Matlab code package. To your question, yes it works in demo mode.
- 2) We have recently implemented an API in Python, which allows to control the device directly. Through external code you can control the machine directly, which enables easier integration into an experimental setup with other devices, allows you to automate the device temperature control based on conditional inputs, connectivity with other devices, while maintaining safety for the patient/test subject. The API can detect TTL signal events, record temperature and timestamp in real time, work with CoVAS and Patient Response Unit (which can also be connected via Python).

Python API

Use of the same command set as MMS

Avoid inactivity timeouts of TCP/IP network socket connection

Set destination temperature and time of each profile segment -ramp, hold and return ramps

Control of each thermode

Safety features maintained



USB



Firmware



Here is the link to the API : https://www.dropbox.com/scl/fi/ytdy9f0xzm9zwpfoqy9b/Python-API_170424.zip?rlkey=eya89g08agt1z2ub5q6d2sdeh&st=t00kti9r&dl=0

The folder contains "examples.py" (Main class) which uses the "tsa_device.py" containing the code to send and receive commands and responses over the serial connection directly with the device. Please also see the readme file in the folder, it instructs on setting the thermode you are using in the regular MMS software in hardware settings before using the API (MMS is only part of the setup, not connected to TSA 2 during use)